TECHNICAL DOSSIER

Montage Studio Video

Architecture, rendering pipeline, data model, security and operations of a local-first video editor for short-form content.

Audience	Software engineers evaluating, operating or extending MSV
Product version	1.3 (development branch after tag `v1.3.00`)
Date	October 2026
Language / runtime	Python 3.11+ (3.12 shipped), FFmpeg 7.1
License	GPL-3.0-or-later
Source of truth	Every figure in this document was read from the code or measured on the reference machine (Intel Core i7-13620H, 16 GB RAM, NVIDIA RTX 4060 Laptop 8 GB, Windows 11).

Contents

1. Executive summary
2. Technology stack
3. Repository layout
4. Architecture
5. Project data model
6. Rendering pipeline
7. Text and subtitles
8. Layers (overlays and animated text)
9. Animated scenes
10. Generated music
11. Voice-over engines
12. Batch production and CLI
13. UI layer
14. Internationalization
15. Error handling
16. Security model
17. Storage, packaging and deployment
18. Testing and CI/CD
19. Performance
20. Extending MSV
21. Known limitations and technical debt
22. Appendices

1 Executive summary

Montage Studio Video (MSV) is a desktop video editor delivered as a local web application. It produces 9:16 Shorts and 16:9 YouTube videos, either interactively or in batches from a CSV file, and runs entirely on the user's machine.

7,831	112	530	4
lines of Python, tests included	automated tests (2 opt-in)	UI strings in 4 languages	OS/Python CI targets

Design principles

- **FFmpeg is the engine.** All decoding, filtering, mixing and encoding runs in FFmpeg subprocesses driven by generated `filter_complex` graphs. Python orchestrates; it never touches video samples, except for the animated scenes it draws itself.
- **Deterministic text.** Titles, subtitles and layers are rasterized with Pillow into transparent PNGs and overlaid by FFmpeg. No dependency on libass or FFmpeg's freetype build; output is identical across Windows, macOS and Linux.
- **A project is a plain dict.** The whole edit is a JSON document merged over defaults. The UI, the batch engine and the CLI all produce the same structure and call the same `render_project()`.
- **Local-first and private.** The server binds to `127.0.0.1`, refuses foreign Host and Origin headers, disables telemetry and never uploads media unless the user picks the ElevenLabs engine.
- **Heavy ML stays isolated.** Voice cloning (PyTorch + Chatterbox) runs in a separate virtual environment and process, so its pinned dependencies cannot break the application.
- **Supply chain discipline.** Shipped dependencies are pinned with SHA-256 hashes, limited to packages published at least two weeks earlier, and audited weekly in CI.

Capabilities in one table

AREA	CAPABILITIES
Editing	Trim, speed (0.1–4×), volume, per-clip crop, 3 fit modes (blur, crop, bars), 8 transitions, fades, silence removal, photo motion (zoom, pan, Ken Burns)
Graphics	Title, 5 subtitle sources with karaoke highlighting, logo watermark, image/video overlays and animated texts with fade/rise/zoom entrances, 7 animated scene templates, automatic intro/ending
Audio	Voice-over track, background music with sidechain ducking, EBU R128 loudness normalization to −14 LUFS, procedurally generated royalty-free music (4 moods)
Voice	Piper TTS (local), ElevenLabs (cloud, word-exact timings), local voice cloning (Chatterbox Multilingual)
Output	MP4 H.264/AAC with `+faststart`, `.srt` sidecar; 9:16, 16:9, 1:1, 4:5; x264 or NVENC / VideoToolbox / Quick Sync with CPU fallback
Automation	CSV or folder batch production, per-row voice generation, templates, command-line interface
Tools	Thumbnail maker, one-click Short, MP3 extraction, Whisper transcription, logo removal (delogo/blur), reversible branded watermark

2 Technology stack

Versions are those pinned in `requirements.lock` for the Windows installer (990 SHA-256 hashes). Developers install the ranges in `requirements.txt` plus `overrides.txt`.

COMPONENT	VERSION	LICENSE	ROLE
Python	3.12 (3.11+ supported)	PSF	Runtime; installed by `uv` for end users
FFmpeg (static, via imageio-ffmpeg 0.6.0)	7.1 GPL build	GPL-3.0	Probe, filters, mixing, encoding (libx264). Drives the project license.
Gradio	6.29.1	Apache-2.0	UI components and event system
FastAPI / Starlette / Uvicorn	0.142.2 / 1.7.0 / 0.54.0	MIT / BSD-3	ASGI host, routing, security middleware
Pillow	13.0.0 (forced by `overrides.txt`)	MIT-CMU	Text rasterization, scene compositing, image fitting
NumPy / pandas	2.5.3 / 3.0.6	BSD-3	Music synthesis, audio analysis, timeline tables
sherpa-onnx	1.13.8 (`<2` enforced)	Apache-2.0	Piper VITS voices, offline
faster-whisper	optional	MIT	Word-level transcription
Chatterbox TTS + PyTorch	0.1.7 + 2.6.0 (CUDA 12.4)	MIT / BSD	Local voice cloning, in a separate virtual environment
Fonts	Montserrat, Anton	SIL OFL 1.1	Shipped typefaces

> **Version constraints worth knowing.** Gradio 6 declares `pillow<13`; MSV overrides it to get 35 security fixes, and the full test suite passes with Pillow 13. sherpa-onnx 2.x removed the built-in espeak-ng phonemizer, which silently produces empty audio for Piper voices; MSV pins `<2` and detects 2.x at runtime. Chatterbox pins `gradio==6.8.0`, `numpy<2` and `torch==2.6.0`, which is why it lives in its own environment.

3 Repository layout

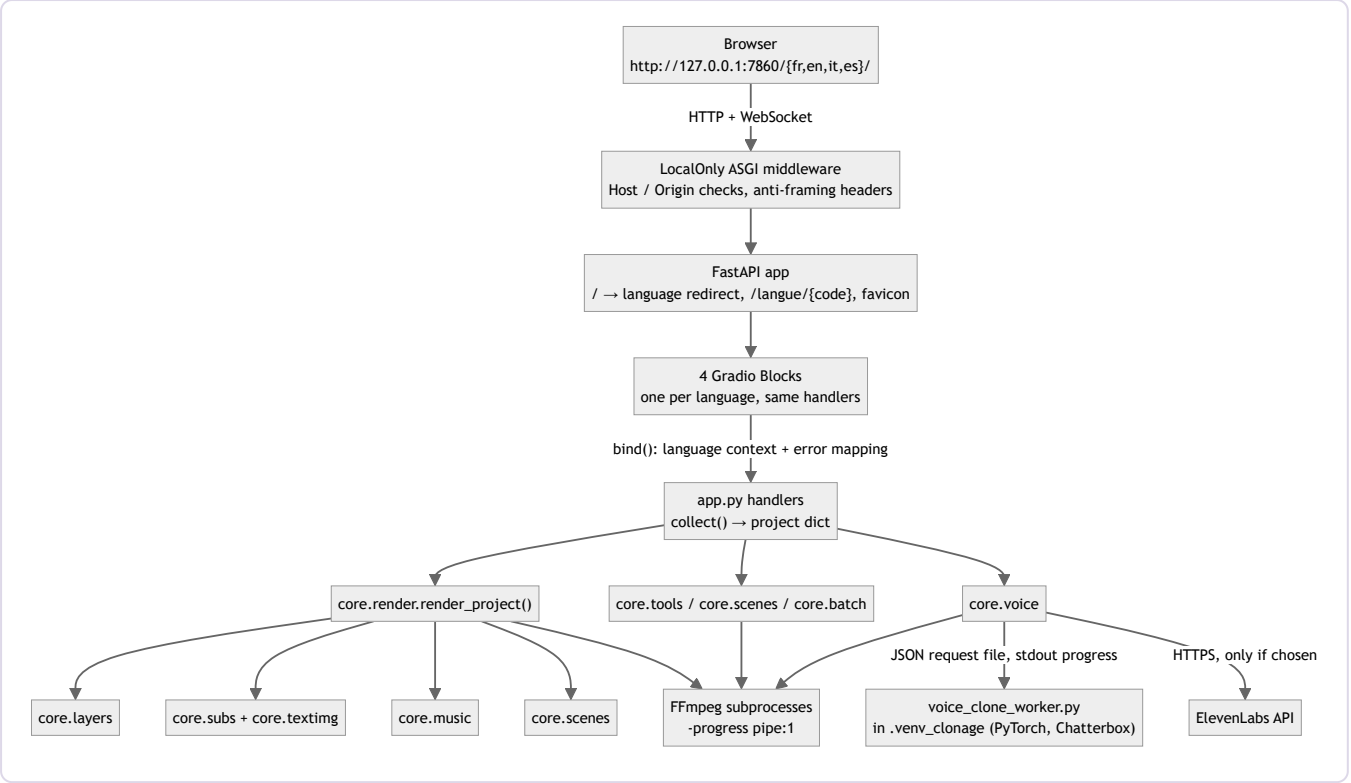
```
montage_studio/                    repository root
├── .github/workflows/
│   ├── ci.yml                     tests (Win/Linux/macOS), pip-audit, installer build + install + uninstall
│   └── release.yml                on tag v*: audit, tests on locked versions, ZIP, Windows installer
├── installer/
│   ├── MontageStudioVideo.iss     Inno Setup script (per-user, no admin rights)
│   └── lancer_msv.bat / .sh       launcher: uv bootstraps Python 3.12 + locked deps on first run
├── outils/maj_securite.py         audits requirements.lock and upgrades only vulnerable packages
├── docs/                          public documentation (GitHub Pages) and PDFs
└── montage_studio/                the application
    ├── app.py                     UI (Gradio Blocks × 4 languages), handlers, ASGI app, security middleware
    ├── cli.py                     command line: rendu | serie | dossier | voix
    ├── core/
    │   ├── render.py              project model + rendering pipeline
    │   ├── ff.py                  FFmpeg discovery, probe, subprocess runner with progress
    │   ├── layers.py              overlays and animated texts → FFmpeg filters
    │   ├── scenes.py              animated scene compositor (Pillow → rawvideo → x264)
    │   ├── textimg.py             text rasterization (titles, subtitles, karaoke)
    │   ├── subs.py                cue model, SRT I/O, script timing, embedded tracks, Whisper
    │   ├── music.py               procedural music synthesis
    │   ├── voice.py               Piper, ElevenLabs, voice cloning orchestration
    │   ├── voice_clone_worker.py  runs inside .venv_clonage (imports nothing from MSV)
    │   ├── batch.py               CSV/folder batch production
    │   ├── tools.py               silence detection, thumbnails, logo removal, time parsing
    │   ├── errors.py              user-facing error explanations + error log
    │   ├── i18n.py                translations (FR source keys → EN/IT/ES) and help pages
    │   ├── paths.py               storage locations per deployment mode
    │   └── version.py             version from VERSION file or git describe
    ├── templates/                 shipped batch templates (JSON)
    ├── exemples/                  CSV examples
    ├── fonts/  assets/            typefaces, logos, icon
    ├── tests/                     pytest suites
    ├── requirements.txt           ranges for developers
    ├── requirements.lock          exact versions + hashes for end users
    ├── overrides.txt              forced versions (Pillow 13)
    └── securite.json              lock policy: exclude-newer date and security exceptions
```

Module size

MODULE	LINES	MODULE	LINES
`app.py`	1,871	`core/textimg.py`	220
`core/i18n.py`	1,359	`core/batch.py`	192
`core/render.py`	761	`core/ff.py`	181
`core/scenes.py`	702	`core/subs.py`	178
`core/voice.py`	430	`core/errors.py`	158
`core/layers.py`	265	`core/paths.py`	108
`core/tools.py`	246	`core/music.py`	104
`tests/` (3 files)	848	`cli.py` , worker, version	208

4 Architecture

A single Python process hosts the web UI and orchestrates FFmpeg subprocesses. Two kinds of work leave the process: FFmpeg commands and, for cloned voices, a worker in a separate interpreter.



Request lifecycle (export)

1. The user clicks *Export video*. Gradio sends the values of every registered field plus the timeline table.
2. The handler is wrapped by `bind(fn, lang)` : it sets the `LANG` context variable so messages come back in the page's language, and converts any exception into a user-facing error (§15).
3. `collect(values)` rebuilds the project dict from the field registry; `clips_from_df()` converts and validates the timeline (§13).
4. `render_project(project, out_path, progress, quality)` runs the pipeline in a temporary work folder and reports progress through Gradio's progress tracker.
5. The handler returns the MP4 path (served from the allowed output folder), the `.srt` and a log.

Core module responsibilities

MODULE	KEY ENTRY POINTS	NOTES
`render`	`merge_defaults` , `render_project` , `parse_crop` , `fit_image`	Owns `DEFAULT_PROJECT` , formats, transitions, quality presets, encoders
`ff`	`ffmpeg_exe` , `probe` , `run_ffmpeg` , `run_capture`	Probe parses `ffmpeg -i` stderr (no ffprobe); results cached by (path, mtime)
`layers`	`build` , `draw_layers` , `norm_overlay` , `norm_text`	Returns extra inputs and filter chains to splice into the final graph
`scenes`	`preview` , `render_scene` , `count_up`	Frame-by-frame Pillow compositor piped to x264
`subs`	`from_script` , `parse_srt` , `write_srt` , `extract_embedded` , `transcribe`	Common cue model with per-word timings

MODULE	KEY ENTRY POINTS	NOTES
`voice`	`generate`, `piper_tts`, `eleven_tts`, `clone_tts_batch`, `install_clone`	Scenes separated by `---`; returns full track, per-scene tracks, SRT
`batch`	`read_csv`, `build_project`, `run_batch`	One failing row never stops the batch
`errors`	`UserError`, `explain`, `user_message`, `log_exception`	Problem + what-to-do messages, rotating error log

5 Project data model

A project is a JSON object. `merge_defaults()` deep-merges it over `DEFAULT_PROJECT`: nested dicts are merged key by key, lists and scalars are replaced. Partial projects (CSV templates, CLI files, tests) are therefore valid.

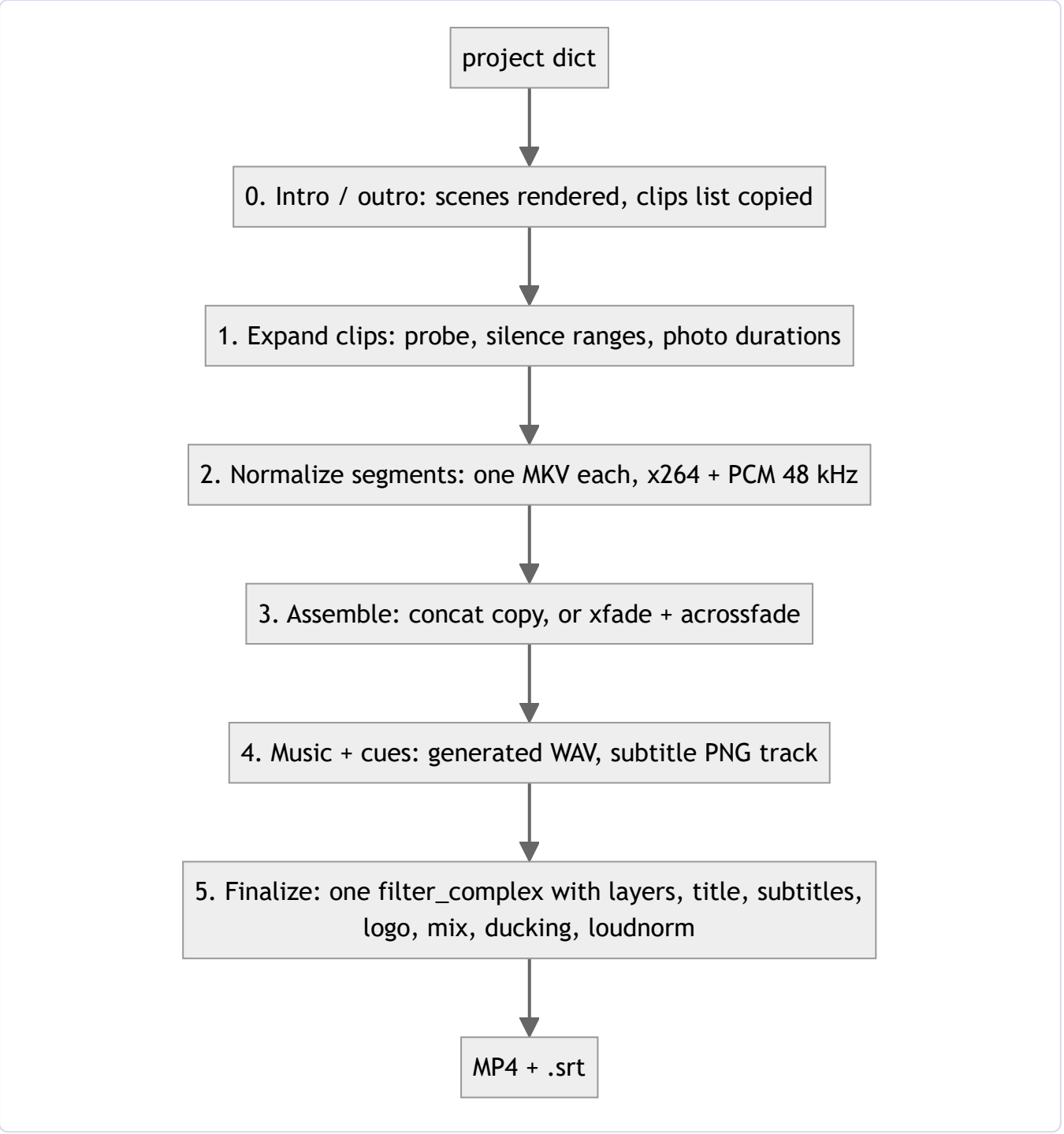
KEY	TYPE / VALUES	MEANING
`format`	`9:16` | `16:9` | `1:1` | `4:5`	1080×1920, 1920×1080, 1080×1080, 1080×1350
`fit`	`flou` | `rogner` | `bandes`	Blurred background, crop to fill, letterbox
`fps`, `quality`, `encoder`	24–60; `aperçu` | `standard` | `haute`; 4 encoders	See §6 presets
`clips[]`	objects	`path`, `start`, `end`, `speed`, `volume`, `mute`, `crop_x` / `crop_y` (0–1), `crop` {left, top, right, bottom in %}, `duration` and `zoom` for images
`transition`	{type, duration}	One of 9 named transitions mapped to FFmpeg `xfade`
`remove_silence`	{enabled, threshold_db, min_silence, padding}	`silencedetect`-based trimming of video clips
`image_zoom`	motion name	Default photo motion; overridable per clip
`voiceover`, `music`	{path, volume, offset} / {path, volume, ducking, auto}	`music.auto` = mood name generates music when no file is set
`title`, `subtitles`, `logo`	objects with `style` dicts	Style keys mirror `textimg.DEFAULT_STYLE`
`overlays[]`, `texts[]`	layer objects	See §8
`intro`, `outro`	scene spec + `enabled`	Rendered at the project format and added to the timeline
`tts`	{enabled, provider, voice, …}	Per-row voice generation in batch mode

> **Enumerations are French identifiers** (`flou`, `fondu enchaîné`, `écran final` …). They are stable keys: the UI shows translated labels through `opts()`, and any value typed in a table in any language is mapped back with `layers.norm_choice()`. Saved projects are therefore language-independent.

Two private keys are set during rendering only: `_intro` and `_outro` (scene durations), used to shift voice-over, title and subtitles. A full example project is given in Appendix B.

6 Rendering pipeline

`render_project()` works in a temporary folder under the private cache and always cleans it up (unless `keep_temp=True`). Each stage is one or more FFmpeg invocations.



Stage 1 — expand clips

- Each clip is probed. Images become one segment of `duration` seconds (default `image_duration`).
- For videos with silence removal enabled, `tools.detect_speech_ranges()` runs `silencedetect` and returns the ranges to keep, padded and merged; each range becomes a segment. Scene clips are excluded (their audio track is silent by design).
- With `fit_images_to_voiceover`, photo durations are recomputed so the photos span the voice-over, accounting for transition overlap and excluding the outro.

Stage 2 — normalize each segment

- Every segment becomes an MKV with identical parameters (resolution, fps, `yuv420p`, x264, PCM s16 48 kHz stereo) so later stages can stream-copy.
- Video: `setpts` for speed, optional `crop` (even dimensions), then the fit graph. The blur fit splits the stream, scales a quarter-size copy, applies `gblur=sigma=10` and darkens it, and overlays the contained image.
- Audio: `atempo` chain (factors kept within 0.5–2.0), volume, `apad`, and 15 ms / 25 ms micro-fades to avoid clicks at cuts.
- Images: fitted once with Pillow, then animated with `zoompan` on a 2× supersampled still to reduce jitter. Motion expressions use cosine easing: `(1-cos(PI*min(1,on/N)))/2`. “Ken Burns” cycles four start/end zoom and focus presets by photo index.

Stage 3 — assemble

- No transition: FFmpeg concat demuxer with `-c copy` (no re-encode).
- Transition: a chain of `xfade` and `acrossfade`. The effective overlap is `max(0.1, min(duration, 0.45 × shortest segment))`; segment start times are derived from it for subtitle re-timing.

Stage 4 — music and subtitle track

- If `music.auto` names a mood and no file is set, `music.generate()` writes a WAV of exactly the assembled duration + 0.5 s, seeded from the project name.
- Cues are rasterized to full-frame PNGs (one per cue, or one per word state in karaoke mode) and listed in an `ffconcat` file with per-image durations and blank gaps. FFmpeg reads it as a variable-frame-rate image stream.

Stage 5 — finalize (single filter graph)

1. Layers (§8) — image/video overlays then animated texts.
2. Title PNG overlay with `enable='between(t,s,e)'`.
3. Subtitle track overlay with `eof_action=pass`.
4. Logo: scaled to a width fraction, opacity via `colorchannelmixer=aa`, 4% margin.
5. Global fade in/out on video and audio.
6. Voice-over: `adelay` + `amix normalize=0`. Music: `-stream_loop -1`, trimmed, faded, then `sidechaincompress` keyed by the speech bus (threshold 0.025, ratio 10, attack 20 ms, release 450 ms).
7. `loudnorm=I=-14:TP=-1.5:LRA=11` (single pass), AAC, `-movflags +faststart`.

Quality presets and encoders

PRESET	SCALE	INTERMEDIATE X264	FINAL X264	AAC
`aperçu` (preview)	0.5	ultrafast, CRF 26	ultrafast, CRF 28	128k
`standard`	1.0	veryfast, CRF 18	fast, CRF 21	160k
`haute` (high)	1.0	veryfast, CRF 16	medium, CRF 18	192k

Hardware encoders replace the final x264 settings: NVENC (`-preset p5 -rc vbr -cq 21`), VideoToolbox (`-b:v 12M`), Quick Sync (`-global_quality 21`). If the hardware encode raises `FFmpegError`, the same command is retried with x264 and a warning is logged. Preview renders always use x264.

Process management

`ff.run_ffmpeg()` launches FFmpeg with `-progress pipe:1`, parses `out_time_us` into a 0–1 fraction, drains stderr on a background thread, and raises `FFmpegError` with the last 2,000 characters of stderr on failure. The FFmpeg binary is resolved from `MONTAGE_FFMPEG`, then imageio-ffmpeg, then the system `PATH`.

7 Text and subtitles

Rasterization

`textimg.render_text()` draws a text block on a transparent W×H canvas: word wrapping to `max_width`, optional rounded box, drop shadow, stroke, and highlight of one word (karaoke) by color or by background pill. Sizes are expressed for a 1080 px short side and scaled by `min(W, H)/1080`, so a style looks the same in preview (half resolution) and final output. Variable fonts (Montserrat) get their weight axis set explicitly.

Cue model

```
{"start": 2.05, "end": 3.03, "text": "Tonight the sky",
 "words": [["Tonight", 2.05, 2.41], ["the", 2.41, 2.55], ["sky", 2.55, 3.03]]}
```

SOURCE	TIMING METHOD
`script`	`estimate_word_times()` spreads words over the voice-over (or the video) with per-word weights based on length and punctuation, then `group_words()` builds cues of N words (max 22 characters).
`srt`	Parsed SRT; word times estimated inside each cue; shifted by the intro duration.
`whisper`	faster-whisper with word timestamps, on the voice-over or on audio extracted from the assembled video.
`vidéo`	Embedded text tracks (`mov_text`, SubRip, ASS…) extracted with `-map 0:s:N -f srt`, track chosen by language, then re-timed per segment for trim, speed, silence removal and transitions. Bitmap tracks (DVD/PGS) are reported as unsupported.

Every render writes the cues to a sidecar `.srt` next to the MP4.

8 Layers (overlays and animated text)

`layers.build()` receives the project, output size, fps, duration and the next free input index, and returns extra FFmpeg inputs plus filter chains spliced into the finalize graph.

LAYER	INPUT	PROCESSING
Image overlay	`-loop 1 -framerate fps -t end -i layer.png`	Resized with Pillow to the requested width (aspect kept, never taller than the frame); opaque images get 5% rounded corners; opacity baked into alpha.
Video overlay (PiP)	`-stream_loop -1 -t dur -i clip` + looped mask PNG	`fps` , `scale` , `alphamerge` with a rounded-rectangle mask, optional opacity, `setpts=PTS-STARTPTS+start/TB` ; muted; loops when shorter than its window.
Animated text	PNG cropped to its bounding box	Rendered with the text engine, then placed by its center in % of the frame and clamped inside it.

Animations as FFmpeg expressions

- **fade**: `fade=t=in:st=s:d=min(0.5, dur/3):alpha=1` and a 0.3 s alpha fade-out.
- **rise**: fade plus `overlay y='y0+rise*pow(1-clip((t-s)/d,0,1),3)'` (ease-out cubic, rise = 6% of frame height).
- **zoom**: fade plus `scale=w='…':h=-2:eval=frame` with an ease-out-back curve from 50% to 100%, and overlay coordinates centered on the layer (`x='cx-w/2'`) because the input size changes every frame.

Every layer overlay also carries `enable='between(t,s,e)'` and `eof_action=pass` . The instant preview (`draw_layers(base, overlays, texts, at)`) composites the final state of the layers visible at time `at` with Pillow, using the same geometry functions as the render.

9 Animated scenes

Scenes are motion-design clips drawn frame by frame in Python, then encoded. They can be inserted manually or attached as project intro/outro.

- **Seven scene types**: title card, phone (media inside a phone frame), end screen (button with click ripple), checklist, key figures (count-up numbers that keep thousands and decimal separators), before/after (wipe), hook question (word-by-word drop).
- **Compositor**: `Scene.frame(t)` paints an animated glow background, then the scene layer, with easing helpers (`ease`, `ease_back`) and LRU-cached text and pill images. A `*word*` syntax colors words with the accent color.
- **Encoding**: frames are written as `rgb24` rawvideo to an FFmpeg pipe (`libx264 -crf 18`) together with a silent `anullsrc` track, so scenes assemble like any other clip.
- **Media**: video media is pre-extracted once at the scene fps, already blur-fitted by FFmpeg, then indexed by frame (looping if short). Previews grab a single frame instead.
- **Responsive layout**: 16:9 uses two columns; portrait stacks. `fitted()` rebuilds the elements at decreasing scale (step 0.08, floor 0.45) until they fit 88% of the height, so long titles never overflow.
- **Resolution scaling**: a `scale` field (0.5 in preview quality) renders scenes at half size for fast previews.

> **Cost.** A 4-second 1080×1920 scene at 30 fps renders in about 6.7 s on the reference machine (≈56 ms per frame, single-threaded Python + Pillow). This is the main CPU hotspot of the product.

10 Generated music

`music.synth(duration, mood, seed)` produces a stereo float signal; `generate()` writes 16-bit 44.1 kHz WAV. No samples or third-party material are involved.

MOOD	TEMPO	CHORDS	ARPEGGIO	RHYTHM
`calme` (calm)	76 BPM	C – G – Am – F	quarter notes	none
`inspirant` (inspiring)	96 BPM	C – Am – F – G	eighth notes	kick on 1 and 3
`énergique` (energetic)	120 BPM	Am – F – C – G	sixteenth notes	kick on every beat, off-beat hi-hat
`suspense`	70 BPM	Dm – B♭ – Gm – A	none	heartbeat double kick, sparse high notes

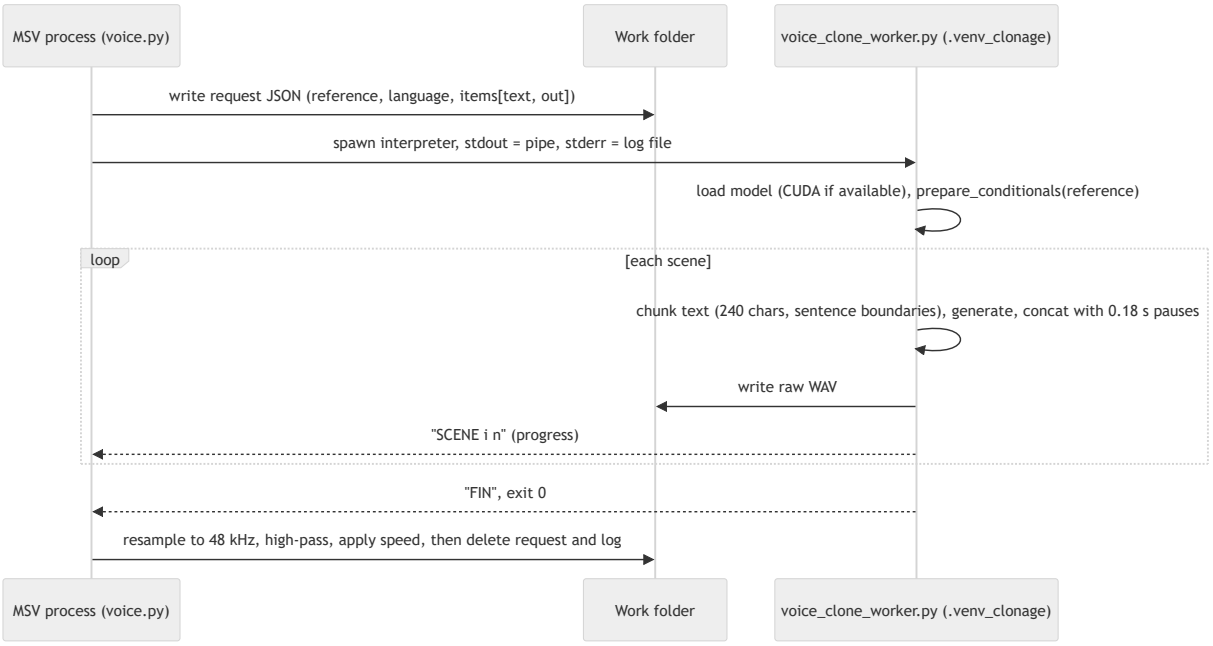
Each bar layers a detuned pad (two voices + octave), a bass note, the arpeggio and percussion; the first bar is pad-only. The seed rotates the starting chord and randomizes arpeggio notes, so videos of a batch differ. The signal gets a 1 s fade-in, a 2.5 s fade-out, `tanh` soft clipping and an 11 ms Haas-style stereo widening. Generating 60 s takes under 1 s; 10 minutes take about 9 s. Final loudness is handled by the render's `loudnorm`.

11 Voice-over engines

`voice.generate(text, out_dir, provider, voice, …)` splits the text into scenes on lines of `---`, synthesizes each scene, concatenates them with a configurable gap, and writes one track per scene, a full 48 kHz track and an SRT.

ENGINE	IMPLEMENTATION	TIMINGS
Piper	sherpa-onnx `OfflineTts` with VITS models downloaded once from the sherpa-onnx GitHub releases (tar.bz2, extracted with path-traversal and link checks, `filter="data"`). Pitch shift via `asetrate` + `atempo`. Only commercially licensed voices are listed (9).	Estimated
ElevenLabs	REST `/v1/text-to-speech/{voice}/with-timestamps` via `urllib`; previous/next text sent for prosody continuity; HTTP errors mapped (401 key, 402 credits, 429 rate limit).	Exact (character alignment → words)
Cloned voice	Chatterbox Multilingual in a separate venv, see below.	Estimated

Voice cloning isolation



- **Why a separate process.** Chatterbox pins versions of gradio, numpy, transformers and torch that conflict with the application. The worker imports nothing from MSV.
- **Deadlock prevention.** The worker's stderr (model download progress bars, warnings) goes to a file, not a pipe. Reading stderr only at exit once filled the OS pipe buffer and hung the worker; this was fixed and is documented in the code.
- **Reference preparation.** `create_clone_voice()` converts the recording to mono 24 kHz, removes leading silence, high-passes at 70 Hz, normalizes to −20 LUFS, and cuts at a pause between 6 and 15 s (the model only conditions on the first seconds). A JSON sidecar records source, date and the consent checkbox.
- **Watermarking.** Chatterbox applies the Perth implicit watermark to every generated waveform.
- **Installation.** `install_clone()` creates the venv and installs `torch==2.6.0` / `torchaudio==2.6.0` from the CUDA 12.4 index, then `chatterbox-tts==0.1.7` (about 5 GB with the 3 GB model downloaded on first use to the Hugging Face cache).

12 Batch production and CLI

CSV batch

- `read_csv()` sniffs the delimiter (`,` `;` tab) on the header line, reads UTF-8 with BOM, normalizes headers (accents removed, lowercase, `_`) and drops empty rows.
- `build_project(template, row, base)` merges the template with the row: media resolution supports `|`-separated lists, folders and glob patterns; relative paths resolve against the base folder.
- Columns: `id, medias, titre, texte, srt, voix_off, musique, debut, fin, recadrage, intro, outro`. `musique=auto[:mood]` enables generated music; `intro` / `outro` take “Title | subtitle” and reuse the template's scene style (or defaults); `non` disables them.
- `run_batch()` optionally generates a voice-over per row from `texte` (Piper, ElevenLabs or cloned voice), renders each row, and isolates failures: an exception becomes a ❌ line in the report and the batch continues.

Command line

```
python cli.py rendu   projets/my_project.json -o sorties/final.mp4 [--qualite aperçu|standard|haute]
python cli.py serie   templates/short_storytelling.json exemples/serie_storytelling.csv --base D:/channel --
sortie sorties/lot1
python cli.py dossier templates/short_clip_parle.json D:/rushes --duree 59 [--titre-nom] --sortie
sorties/clips
python cli.py voix    script.txt --moteur piper|elevenlabs
```

Shipped templates: `short_clip_parle` (filmed talking-head Short), `short_storytelling` (photos timed to narration), `short_enfants` (toddler content), `video_youtube_16x9`.

13 UI layer

`app.py` builds the same Gradio Blocks four times (one per language) and mounts them under `/fr`, `/en`, `/it`, `/es` of one FastAPI app. `/` redirects to the language stored in a `SameSite=Strict`, `HttpOnly` cookie or negotiated from `Accept-Language`.

Field registry: UI ⇄ project JSON

- Every setting component is registered with `reg("dotted.key", component)`, for example `reg("subtitles.style.size", Slider(...))`. The ordered registry defines the input list of every action.
- `collect(values)` walks the registry, applies conversions (uploaded files are persisted into the media folder; untouched textboxes that Gradio sends as `None` become `""`; dataframes become lists), sets values with `set_path`, and returns `merge_defaults(project)`.
- `project_values(project)` does the reverse for loading projects and templates.
- Complex values (intro/outro scene specs) live in registered `gr.State` components; the layer tables are registered dataframes with converters (`TABLES`).

Timeline and validation

The timeline is an editable dataframe with 10 columns read by position (headers are translated). `clips_from_df()` sorts by order, converts numbers, parses crops and per-photo motion, and calls `check_clip()`, which raises `UserError` for semantic problems: missing file, audio in the timeline, speed outside 0.1–4, negative volume, start beyond the file's duration, end before start.

Previews

- **Instant preview** (no FFmpeg render): base frame of the clip active at the requested time (`clip_at()`), plus layers visible at that time, title if active, and a sample subtitle with the current style.
- **Quick preview**: full pipeline at half resolution with ultrafast presets.

14 Internationalization

- **Gettext-style with French source keys.** `_("Texte", **fields)` looks up the active language from a `contextvars.ContextVar` and formats placeholders. Missing keys fall back to French.
- **One table, four columns.** `_STRINGS` holds 530 tuples (FR, EN, IT, ES); `HELP` holds one Markdown help page per language.
- **Context per request.** Gradio runs handlers in worker threads; `bind(fn, lang)` sets and resets `LANG` around each call, so the same handler serves all four pages.
- **Choice widgets.** `opts(values)` returns `(translated label, French value)` pairs. `translations(text)` lists all translations of a key, which `norm_choice()` uses to accept values typed in any language.
- **Guard rails in tests.** Every tuple must have four non-empty entries with identical `{placeholders}`, and a source scan checks that every literal passed to `_()` in the code exists in the table.

15 Error handling

Users never see a traceback. Every UI action is wrapped by `bind()`, which converts any exception into a `gr.Error` built by `errors.user_message()`: problem, what to do, short technical detail, and the path of the error log.

FAMILY	DETECTED FROM	EXAMPLE ADVICE
Syntax	`JSONDecodeError` (line/column), `csv.Error`, `UnicodeDecodeError`, parse errors of time/crop	Save the CSV as “CSV UTF-8” with a header row
Semantics	`UserError` raised by validators (timeline, layer times)	The end must be greater than the start
FFmpeg	stderr patterns: invalid data / moov atom, no such file, permission denied, no space, NVENC/CUDA/QSV, out of memory	Choose the “cpu (x264)” encoder
System	`FileNotFoundError`, `PermissionError`, `ENOSPC`, `MemoryError`, network errors, `ModuleNotFoundError`	Exact `pip install` command for the missing module
Voice	empty Piper audio, `wave.Error`, sherpa-onnx 2.x	Reinstall `"sherpa-onnx<2"`
Bug	anything else (`KeyError`, `TypeError` …)	“Not your fault”; send `journal_erreurs.log`

Unexpected exceptions are appended with their full traceback to `journal_erreurs.log` in the data folder, rotated to `.ancien.log` beyond 512 KB. Expected `UserError`s are not logged. Messages are HTML-escaped before display.

16 Security model

Threat model: a local web application reachable from the user's own browser, where any website the user visits could try to reach `127.0.0.1`.

THREAT	CONTROL
Remote access over the network	Uvicorn binds to `127.0.0.1` only.
DNS rebinding (a site resolving its own domain to 127.0.0.1)	`LocalOnly` ASGI middleware rejects any `Host` other than `127.0.0.1` / `localhost` (HTTP 403, WebSocket close 1008).
Cross-site requests (CSRF) from a malicious page	Same middleware rejects any `Origin` that is not local, including `null` and look-alikes such as `127.0.0.1.evil.com` (parsed with `urlsplit`).
Clickjacking	`X-Frame-Options: DENY` and `CSP frame-ancestors 'none'` on every response; plus `nosniff` and `no-referrer`.
Arbitrary file download through Gradio's file route	`allowed_paths` limited to outputs, media, projects and examples; `blocked_paths` covers code, both venvs, assets, cache, templates, cloned voices, top-level files and the private folder.
Data exfiltration / telemetry	`GRADIO_ANALYTICS_ENABLED=False` and `HF_HUB_DISABLE_TELEMETRY=1` set before imports. No network calls unless ElevenLabs or a first-time model download is used.
Malicious model archives	Piper tarballs rejected if any member escapes the target folder or is a link; Python's `data` extraction filter applied.
Secrets	ElevenLabs key stored only in the private folder (`voix_config.json`, git-ignored, download-blocked) or read from `ELEVENLABS_API_KEY`; password-type input in the UI.
Supply chain	`requirements.lock` with SHA-256 hashes; packages restricted to those published ≥ 2 weeks before the lock date (`securite.json`), with dated exceptions only for security fixes; `pip-audit` on every push and weekly; GitHub Actions pinned by commit SHA; `uv` binary verified by SHA-256 in CI and releases.
Misuse of voice cloning and logo removal	Mandatory consent checkbox and consent record; inaudible watermark on cloned audio; rights notice on the logo-removal tool.

Security behavior is covered by tests: foreign Host/Origin variants return 403, anti-framing headers are present, and source files under the app folder cannot be fetched through Gradio's file route.

17 Storage, packaging and deployment

Storage modes (`core/paths.py`)

MODE	TRIGGER	USER DATA	PRIVATE DATA
development	default (git checkout)	app folder (`medias/` , `sorties/` , `projets/`)	app folder
installed	`MSV_INSTALLED=1` (set by the launcher)	`Documents/Montage Studio Video` (OneDrive redirection and XDG handled)	`%LOCALAPPDATA%\MontageStudioVideo` , `~/Library/Application Support/…` , `~/.local/share/…`
custom	`MSV_DATA_DIR=<folder>`	that folder	`<folder>/.msv`

Windows installer

- Inno Setup, per-user install into `%LOCALAPPDATA%\Programs\Montage Studio Video` , no administrator rights, x64, Windows 10+.
- The installer ships the application code, the `uv` binary and `requirements.lock` only. On first launch, `lancer_msv.bat` uses `uv` to download a managed Python 3.12 and install the locked, hash-verified packages into `%LOCALAPPDATA%\MontageStudioVideo\venv` (≈400 MB, once).
- On later launches the launcher compares the shipped lock file with the copy stored in the venv; any difference triggers a clean rebuild, which makes upgrades deterministic.
- Uninstall removes the program and private data but never the user's Documents folder (verified in CI).

Versioning

`core/version.py` reads a `VERSION` file written by the release pipeline, or falls back to `git describe --tags --always` , or `dev` .

18 Testing and CI/CD

SUITE	TESTS	COVERAGE
`tests/test_app.py`	91	Real renders (Short, transitions, crop, embedded subtitles, layers with pixel checks, photo motions, intro/outro + generated music in a CSV batch), all scenes × all formats, UI helpers, language pages, security guard, translation completeness, error explanations, time converter, logos
`tests/test_voix.py`	15	Scene splitting, ElevenLabs against a local fake HTTP server (voices, invalid key, timings, batch), voice replacement, clone reference preparation and chunking, translation scan; Piper and real cloning are opt-in
`tests/test_installation.py`	6	Storage modes, user templates priority, version file, lock file consistent with requirements

Opt-in tests that download models: `MSV_TEST_PIPER=1` and `MSV_TEST_CLONAGE=1`. Run everything with `python -m pytest tests` from `montage_studio/` (about 25 s on the reference machine).

Pipelines

- **ci.yml** (push, pull request, weekly): tests on Windows/Python 3.12, Ubuntu/3.11, Ubuntu/3.12, macOS/3.12; `pip-audit` on the lock file and on a fresh developer install; Windows installer job that compiles, installs silently, checks the installed file set, runs the first-launch bootstrap, imports the app in installed mode, then uninstalls and checks that user files survive.
- **release.yml** (tag `v*`): audit and tests against the exact locked versions, source ZIP built with `git archive`, GitHub release, Windows installer attached.

19 Performance

Measured on the reference machine (Core i7-13620H, 16 GB, RTX 4060 Laptop 8 GB, Windows 11, FFmpeg 7.1, x264 encoder unless stated).

OPERATION	OUTPUT	WALL TIME
Full Short, high quality: intro + 2 clips + Ken Burns photo + title + karaoke + 2 layers + outro + generated music	14.5 s, 1080×1920	15.5 s
Quick preview (half resolution), 3 clips + layers + generated music	12 s, 540×960	2–9 s (warm / cold)
Quick preview with intro and outro scenes	9 s, 540×960	4 s
Animated scene	4 s, 1080×1920	6.7 s
Scene preview frame	1 image	0.1–0.2 s
Generated music	60 s / 10 min	< 1 s / 8.7 s
Cloned voice, 2 scenes (CUDA), model load included	12.5 s of audio	47 s
Test suite	112 tests	≈ 25 s

Where the time goes

- Segment normalization re-encodes every clip once; with many short clips the per-process overhead of FFmpeg dominates.
- Scenes are CPU-bound Python (Pillow compositing per frame, single thread).
- The finalize stage decodes one stream per layer and per subtitle track; dozens of layers increase cost linearly.
- GPU encoders accelerate only the final encode, not filtering.

20 Extending MSV

Add a transition

Add `"display name": "xfade_name"` to `render.TRANSITIONS` and a translation tuple for the display name. Any FFmpeg `xfade` transition works.

Add a photo motion

Extend `render.MOTIONS` and `_motion()`, which returns zoompan expressions `(z, x, y)`.

Add an animated scene

1. Write a method `Scene.my_scene(self, layer, u)` that draws on `layer` at time `u`, using `rise`, `place`, `text` and `fitted`.
2. Register it in `SCENES` and in the dispatch dict of `Scene.frame()`.
3. Add its description, optional example lines and translations in `app.SCENE_INFO` / `SCENE_EXAMPLES` and `i18n`.
4. The parametrized test `test_apercu_des_scenes` covers it in all four formats automatically.

Add a voice engine

Implement a function returning word timings, branch on the provider in `voice.generate()`, extend `ENGINES` and `voice_settings()` in `app.py`, and pass its options through the batch `tts` dict.

Add a UI setting

Add the key to `DEFAULT_PROJECT`, register the component with `reg("dotted.key", …)` before `vals` is computed in the Editor tab, and read it in the pipeline. Saving, loading, templates and batch mode pick it up automatically.

Add a language

Add the code to `LANGS`, a fifth column to every `_STRINGS` tuple and the unpacking loop, a `HELP` page, and Inno Setup messages. The completeness test enforces the rest. This is the most invasive extension (see §21).

21 Known limitations and technical debt

ITEM	IMPACT	SUGGESTED DIRECTION
Table-based editing, no graphical timeline or live playback	Slower for complex edits	Custom front-end component (Svelte/React) on top of the same JSON model
`app.py` mixes UI layout, handlers and the ASGI app (1,871 lines)	Harder to review and test	Split into `ui/` (layout per tab), `handlers/`, `server.py`
French identifiers and enum values in code and JSON	Learning curve for non-French contributors	Keep JSON values (compatibility) but add English aliases and docstrings
i18n stored as 4-tuples in a Python file	Adding a language touches 530 entries	Move to per-language JSON or gettext `.po` files
Probe parses `ffmpeg -i` stderr	Sensitive to FFmpeg message changes	Use `ffprobe -of json` (ship ffprobe)
Scenes rendered single-threaded in Python	Main CPU hotspot (~56 ms/frame at 1080p)	Render frames in a process pool, or cache static layers per scene
Single-pass `loudnorm`	Small deviation from target loudness	Two-pass measurement for final exports
Voice cloning requires a 5 GB separate environment; CUDA 12.4 wheels only tested on Windows/NVIDIA	Heavy; slow on CPU; macOS untested	Optional download with CPU/MPS variants; ONNX export
No static type checking or linting in CI	Regressions caught only by tests	Add ruff and mypy jobs
Installer not code-signed	SmartScreen warning	Authenticode signing in the release job
Legacy `DOCUMENTATION.md` embeds old source code	Out of date	Replace with generated API docs; this dossier supersedes it

22 Appendices

A. Environment variables

VARIABLE	EFFECT
`MSV_DATA_DIR`	Custom storage mode (portable install, tests, demos)
`MSV_INSTALLED=1`	Installed storage mode (set by the launcher)
`MONTAGE_FFMPEG`	Path of the FFmpeg binary to use
`ELEVENLABS_API_KEY`	ElevenLabs key (else the stored key)
`MSV_TEST_PIPER=1`, `MSV_TEST_CLONAGE=1`	Enable model-downloading tests
`MSV_PREPARER_SEULEMENT=1`	Launcher: install dependencies and exit (used by CI)
`GRADIO_ANALYTICS_ENABLED`, `HF_HUB_DISABLE_TELEMETRY`	Forced off by the application

B. Example project (JSON)

```
{
  "name": "demo", "format": "9:16", "fit": "flou", "quality": "haute",
  "clips": [
    {"path": "medias/sunset.mp4", "start": 0, "end": 5},
    {"path": "medias/mountain.jpg", "duration": 4, "zoom": "Ken Burns"}
  ],
  "transition": {"type": "fondu enchaîné", "duration": 0.5},
  "intro": {"enabled": true, "scene": "carte titre", "duration": 3,
            "chip": "Episode 3", "title": "The *secret* of the frozen lake"},
  "outro": {"enabled": true, "scene": "écran final", "duration": 4,
            "title": "Subscribe for *more*", "button": "Subscribe", "button_done": "Subscribed"},
  "title": {"text": "A magical sunset", "start": 0, "end": 2.5},
  "subtitles": {"source": "script", "karaoke": true,
                "script": "Tonight the sky caught fire above the mountains"},
  "texts": [{"text": "Subscribe!", "start": 7, "end": 11, "y": 88, "box": true, "animation": "montée"}],
  "overlays": [{"path": "medias/logo.png", "start": 3, "x": 85, "y": 7, "width": 14, "animation": "fondu"}],
  "music": {"auto": "inspirant"}
}
```

C. Final FFmpeg filter graph of a real render

Generated by `render_project()` for a preview with one zooming image layer, one rising text, a title, script subtitles and generated music (inputs: 0 assembled video, 1–2 layer PNGs, 3 title, 4 subtitle concat track, 5 music).

```
[1:v]format=rgba,fade=t=in:st=0.500:d=0.500:alpha=1,fade=t=out:st=5.200:d=0.300:alpha=1,
     scale=w='max(2,trunc(74*(0.5+0.5*(1+2.70158*pow(clip((t-0.500)/0.500,0,1)-1,3)
     +1.70158*pow(clip((t-0.500)/0.500,0,1)-1,2)))/2)*2)':h=-2:eval=frame[ca0];
[0:v][ca0]overlay=x='459.0-w/2':y='77.0-h/2':enable='between(t,0.500,5.500)':eof_action=pass[cl0];
[2:v]format=rgba,fade=t=in:st=1.000:d=0.500:alpha=1,fade=t=out:st=5.200:d=0.300:alpha=1[ca1];
[cl0][ca1]overlay=x=150:y='455+57*pow(1-clip((t-
1.000)/0.500,0,1),3)':enable='between(t,1.000,5.500)':eof_action=pass[cl1];
[3:v]format=rgba[tt];
[cl1][tt]overlay=0:0:enable='between(t,0.000,2.000)'[vt];
[4:v]format=rgba[st];
[vt][st]overlay=0:0:eof_action=pass[vs];
[vs]format=yuv420p[vout];
[5:a]aresample=48000,aformat=channel_layouts=stereo,atrim=0:5.500,volume=0.180,
     afade=t=in:d=1,afade=t=out:st=4.125:d=1.38[mus];
[0:a]asplit=2[sp1][sp2];
[mus][sp2]sidechaincompress=threshold=0.025:ratio=10:attack=20:release=450[mduck];
[sp1][mduck]amix=inputs=2:duration=first:normalize=0[mx];
[mx]loudnorm=I=-14:TP=-1.5:LRA=11,aresample=48000[aout]
```

D. Getting started for developers

```
git clone <repository> && cd montage_studio/montage_studio
python -m venv .venv && .venv\Scripts\activate          # macOS/Linux: source .venv/bin/activate
pip install -r requirements.txt pytest && pip install --no-deps -r overrides.txt
python -m pytest tests -q                               # ≈ 25 s
python app.py                                           # http://127.0.0.1:7860
pip install faster-whisper                              # optional: Whisper subtitles
```